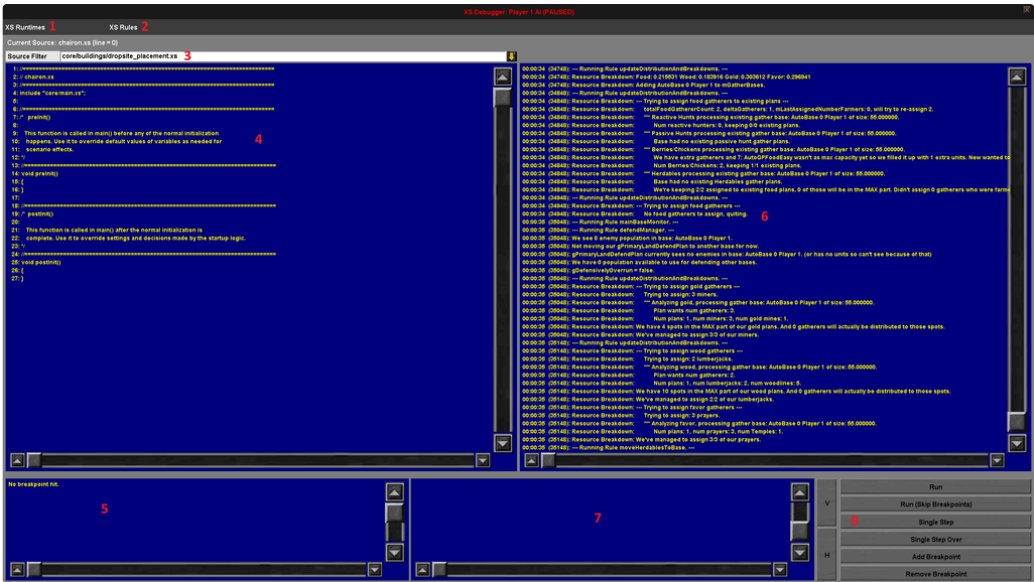


XS debugger

The in-game XS debugger is available for all XS runtimes. It functions like any other source debugger, although being rather rudimentary.

The XS Debugger is enabled if any of these three configs is enabled: *AIDebug*, *DebugTriggers*, *DebugRandomMaps*.

The debugger can open automatically during certain situations if certain configs are defined. It can be manually opened by **alt-shift-d** (default binding). Remapping this hotkey can be done in user.con: map ("alt-shift-d", "game", "toggleXSDebugger")



1. XS Runtimes

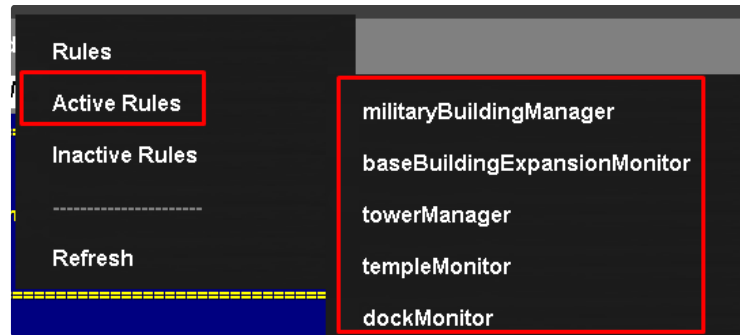
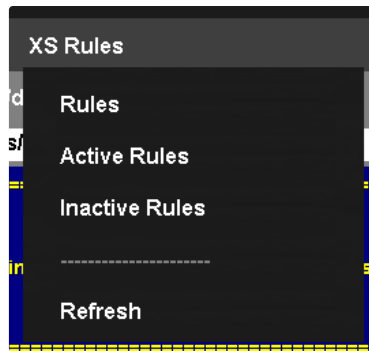
All XS scripts that are being executed are listed here in a dropdown list.



Via here you can switch what code you want to inspect inside the debugger.

2. XS Rules

All the rules that are contained with the select XS script are displayed here. You can also see here which of the rules are active/inactive.



3. File browser

Usually an XS script is made up from multiple files that are linked together via includes. Changing what specific file you want to inspect can be done here.

4. XS code from file

Displays all the code from the select XS files, this is also where you interact with breakpoint which are explained later.

5. Callstack

Only active while hitting a breakpoint. Displays the code path that got you to the point you're currently analyzing.

Example: function1 calls function2 calls function3 and we've put our breakpoint in function3, then we will see that we were called by function2/1.

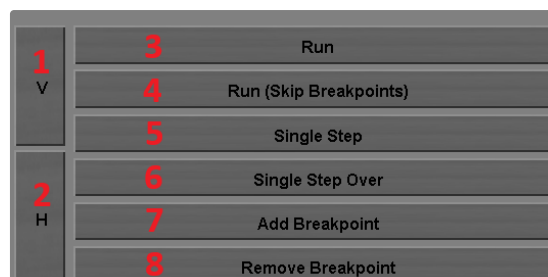
6. Output

Displays all output from the XS script. Think aiEcho / trEcho / rmEchoInfo (and more) calls.

7. Local/global variables

Shows the values of certain variables. What variables are displayed is explained in section 8.1/8.2.

8. Interactions



8.1 Local variables

Switch to showing only variables that are defined in the current function you're debugging in the 7. Local/global variables window.

8.2 Heap/global variables

Switch to showing all global variables in the 7. Local/global variables window.

8.3 Run

Stop debugging and run the XS code again until another breakpoint is hit. You **MUST** do this before you unpause the game again or the script that had a breakpoint will perpetually be stuck in that breakpoint.

8.4 Run (Skip Breakpoints)

Stop debugging and the XS code again. If ever a breakpoint is hit again the XS debugger will be brought up but the breakpoint is instantly skipped for the rest of the game.

Clicking this button again undoes the automatic breakpoint skipping.

8.5 Single Step

F11: is a hardcoded hotkey to trigger this.

Advances the code by one step, and potentially goes inside another function if that line of code was calling a function.

8.6 Single Step Over

F10: is a hardcoded hotkey to trigger this.

Advances the code by one step but doesn't leave the current function, executes function calls without showing the code.

8.7 Add Breakpoint

You can click on a line in the "XS code from file" part of the debugger to select it. This selection has no visual indicator. Subsequently pressing "Add Breakpoint" makes the selected line go red. This means that if this line were to be executed the XS Debugger will open at this spot and you can step through the code.

```
9: //=====
10: // calculateFoodDropSitePlacement
11: //=====
12: void calculateFoodDropSitePlacement(int buildPlanID = -1, int bplD = -1, int kbResourceID = -1)
13: {
14:     if (kbResourceGetIsIDValid(kbResourceID) == false) ←
15:     {
16:         aiEchoWarning(aiPlanGetName(buildPlanID) + ", calling calculateFoodDropSitePlacement with kbResourceID being -1, placement " +
17:             "can't function.");
18:         return;
19:     }
```

After actually hitting the breakpoint the line will turn light blue instead. That blue line indicates where the code is currently at with executing.

```
9: //=====
10: // calculateFoodDropSitePlacement
11: //=====
12: void calculateFoodDropSitePlacement(int buildPlanID = -1, int bplD = -1, int kbResourceID = -1)
13: {
14:     if (kbResourceGetIsIDValid(kbResourceID) == false) ←
15:     {
16:         aiEchoWarning(aiPlanGetName(buildPlanID) + ", calling calculateFoodDropSitePlacement with kbResourceID being -1, placement " +
17:             "can't function.");
18:         return;
19:     }
```

8.8 Remove Breakpoints

Click on the line you want to remove a breakpoint from and then click the button.